



Memory Controller

Memory Mapping · Memory Protection · Data-Width Adaptation

Grundlagenpraktikum Rechnerarchitektur (GRA) — Final Project

Niklas · Ahmed · Andrei

Agenda

Niklas

1 Problem Statement & Literature research

Niklas

2 Architecture

Ahmed

3 Implementation — Frontend (C)

Ahmed

4 Implementation — SystemC Module

Andrei

5 Correctness & Testing

Ahmed

6 Evaluation & Integration

Andrei

7 Summary & Outlook

Niklas



What is memory?



Literature research

von Neumann vs. Harvard & Memory Mapping

Ahmed

von Neumann architecture

Program and data share memory and bus → accesses happen sequentially.

Harvard architecture

Physically separate memories → parallel access is possible, but the hardware is more complex [1].

Our memory controller

ROM and RAM share the same address space
→ corresponds more closely to a von Neumann architecture.

Sources: [1] Leitenberger — Harvard/von Neumann architecture [2] Wikipedia — Memory-mapped I/O

Memory Mapping

Ahmed

- Mapping of logical addresses onto physical memory regions or peripherals
- e.g. memory-mapped I/O: peripheral devices (graphics cards, etc.) are addressed via memory addresses instead of dedicated I/O instructions.
- Pointer-based access simplifies usage in high-level languages such as C [2].



Literature research

Ahmed

Memory Protection Unit & Data-Width Adaptation



Memory Protection Unit

Prevents unauthorized memory accesses. In this project, realized via an owner assignment per memory block (owners map).



Data-Width Adaptation

Adjusts the access width to 1 or 4 bytes. The main memory itself only supports 4-byte accesses. The memory controller performs the adaptation.

Takeaway

All three functions (mapping, protection, data-width adaptation) are often implemented as separate modules in practice. For this project, they are deliberately combined into a single memory controller module.

Sources: [1] Leitenberger — Harvard/von Neumann architecture [2] Wikipedia — Memory-mapped I/O

Problem Statement

Niklas

Why do we need a memory controller?

Memory access is central, but risky

- Especially in von Neumann architectures, program and data share the same address space
- Errors or unauthorized access can compromise the whole system.

Project goal

- SystemC module that coordinates memory accesses
- maps ROM and RAM regions
- Enforces access rights
- Supports flexible access widths.



Memory Mapping

Address space is mapped onto ROM and RAM regions.



Memory Protection

Access rights are managed per user, block by block.



Data-Width Adaptation

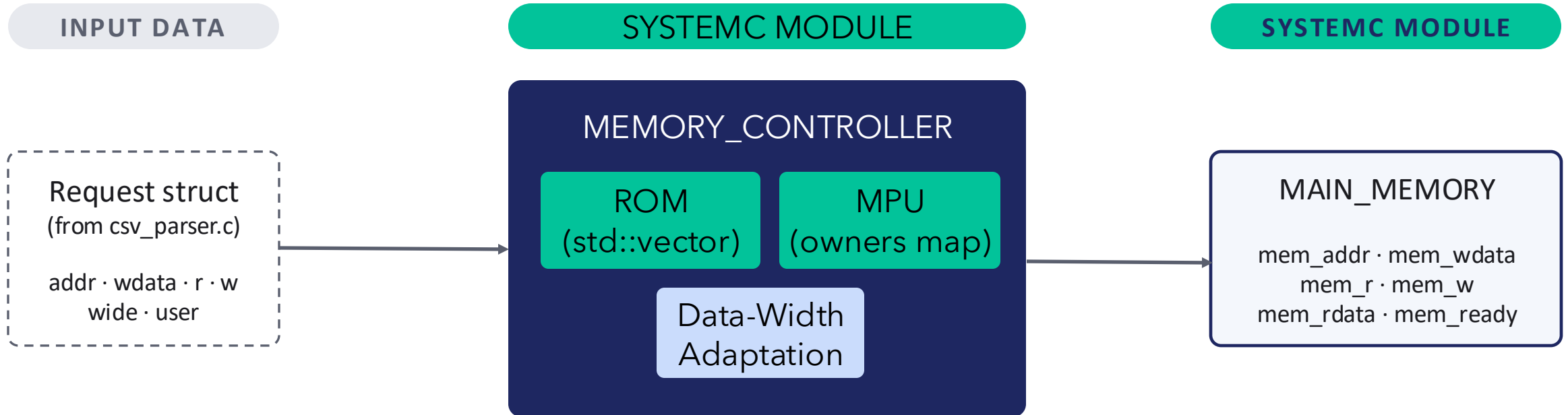
1-byte or 4-byte accesses, independent of the main memory's native width.



Architecture Overview

Niklas

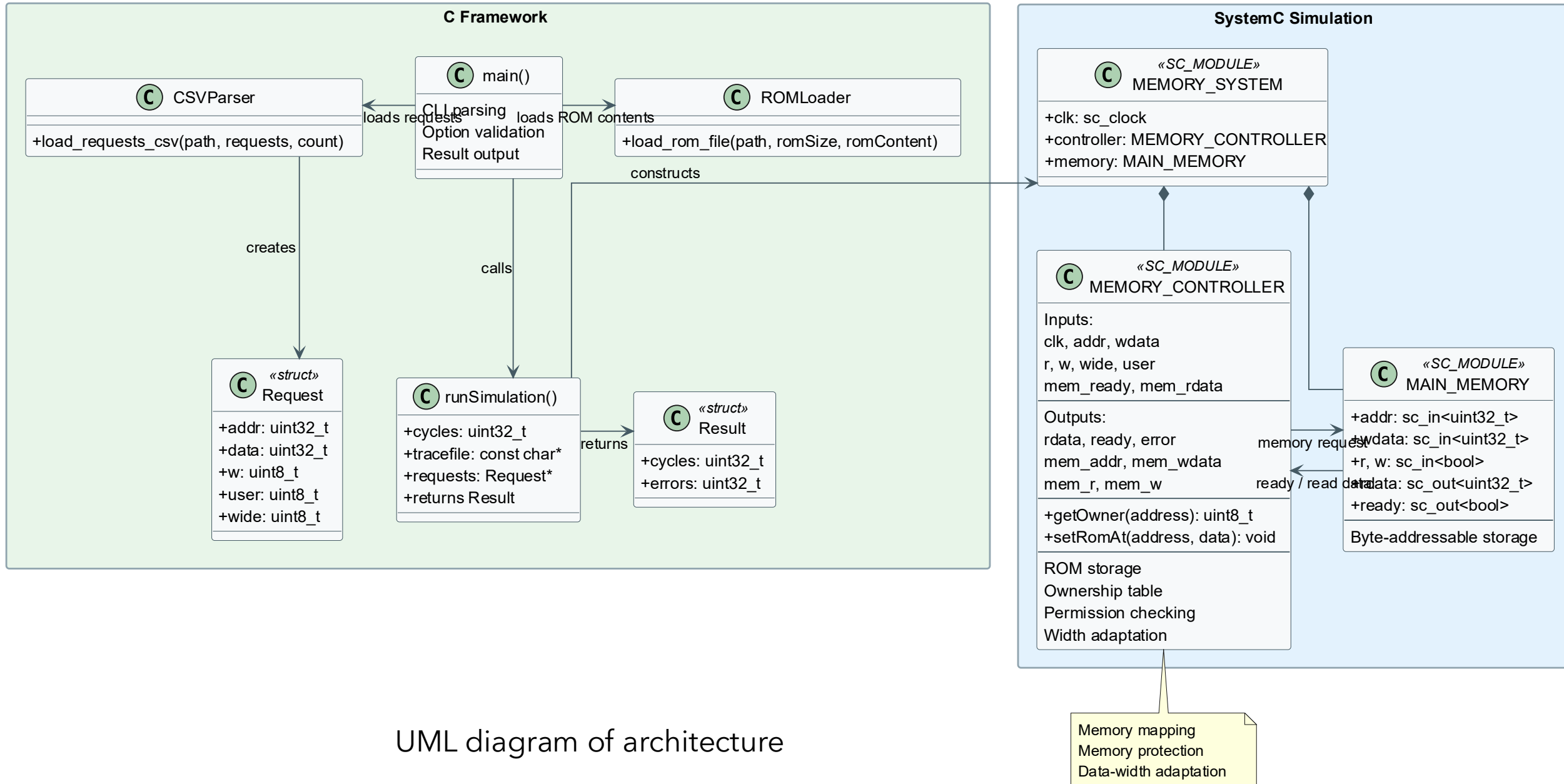
Two communicating SystemC modules



clk (sc_in<bool>) drives both modules via SC_THREAD, sensitive << clk.pos()

Address range [0, rom-size) → ROM
Rest → MAIN_MEMORY (after permission check)

Memory Controller — Project Architecture



UML diagram of architecture

Implementation — Frontend (C)

Ahmed

Parsing, validation, CLI



csv_parser.c

- Reads the request file line by line
- Validates every field: type (R/W), address, data, user (0–255), wide (T/F)
- Produces an array of Request structs



rom_loader.c

- Reads 32-bit values (dec./hex.) into an array
- Missing values → 0
- Too many values → error + abort



main.c

- CLI parsing via getopt_long
- --cycles, --tf, --latency-rom, --rom-size, --block-size, --rom-content
- Validation: power of two, block-size > 0, file access

Secure programming: no unsafe functions (e.g. atoi), memory is consistently released (free(romContent), free(requests)), error messages go to stderr.

Correctness — Test Methodology

Ahmed

Targeted CSV test scenarios per edge case

Each scenario is simulated for a fixed number of cycles; cycles/errors from the Result struct are compared against the expected value (automated via the Makefile test target).

Test case	Edge case covered	Exp. errors	Result
rom_read	Regular ROM read access	0	passed
rom_write_error	Write attempt to ROM	1	passed
ram_wide	4-byte accesses in RAM	0	passed
byte_unaligned	1-byte accesses at offset addresses	0	passed
block_boundary_permissions	Permission check across block boundaries	1	passed
user255_release_read	Block release by user 255	0	passed
rom_ram_boundary_error	Access spanning the ROM/RAM boundary	1	passed
smoke	Multiple mixed requests	2	passed
permissions	Access rights of different users	1	passed



Implementation — SystemC Module

Andrei

State flow inside behavior()

- 1 wait() on the clock edge; read r/w , if no request → keep waiting
- 2 ready = 0; latch addr/wdata/wide/user
- 3 Error checks: r&&w simultaneously, address overflow, ROM/RAM boundary crossed
- 4 ROM address? → only reads allowed, wait latencyRom cycles, set rdata
- 5 RAM address? → check hasAccess(), update ownership if needed, memoryRead/-Write via mem_ signals
- 6 1-byte write: read the full 4-byte value first, replace the LSB, write back (data-width adaptation)
- 7 ready = 1 (error = 1 on access violations, otherwise 0)

Trick: rangeOverflows checks: $bytes > 0 \ \&\& \ start > UINT32_MAX - (bytes - 1)$ --> detects an address overflow without overflowing itself.
Normal: $start + bytes > UINT32_MAX$ --> can create overflow if start is close to $UINT32_MAX$



Walkthrough Example

Andrei

Request: W, 0x100, 0x11, 1, F

1

csv_parser.c

creates a Request struct from the line

2

simulation.cpp

drives addr, wdata, user, wide onto the signals, sets w

3

behavior()

detects the write request, sets ready = 0, checks access rights on the block

4

1-byte access

the full 4-byte value is read from MAIN_MEMORY

5

Data-Width Adaptation

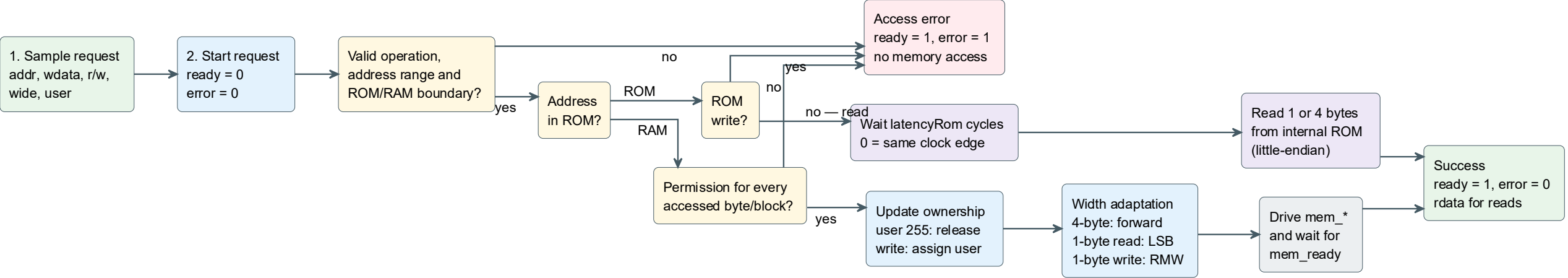
the least-significant byte is replaced, the remaining 3 bytes are preserved

6

Completion

the result is written back, ready = 1

Memory Request Processing

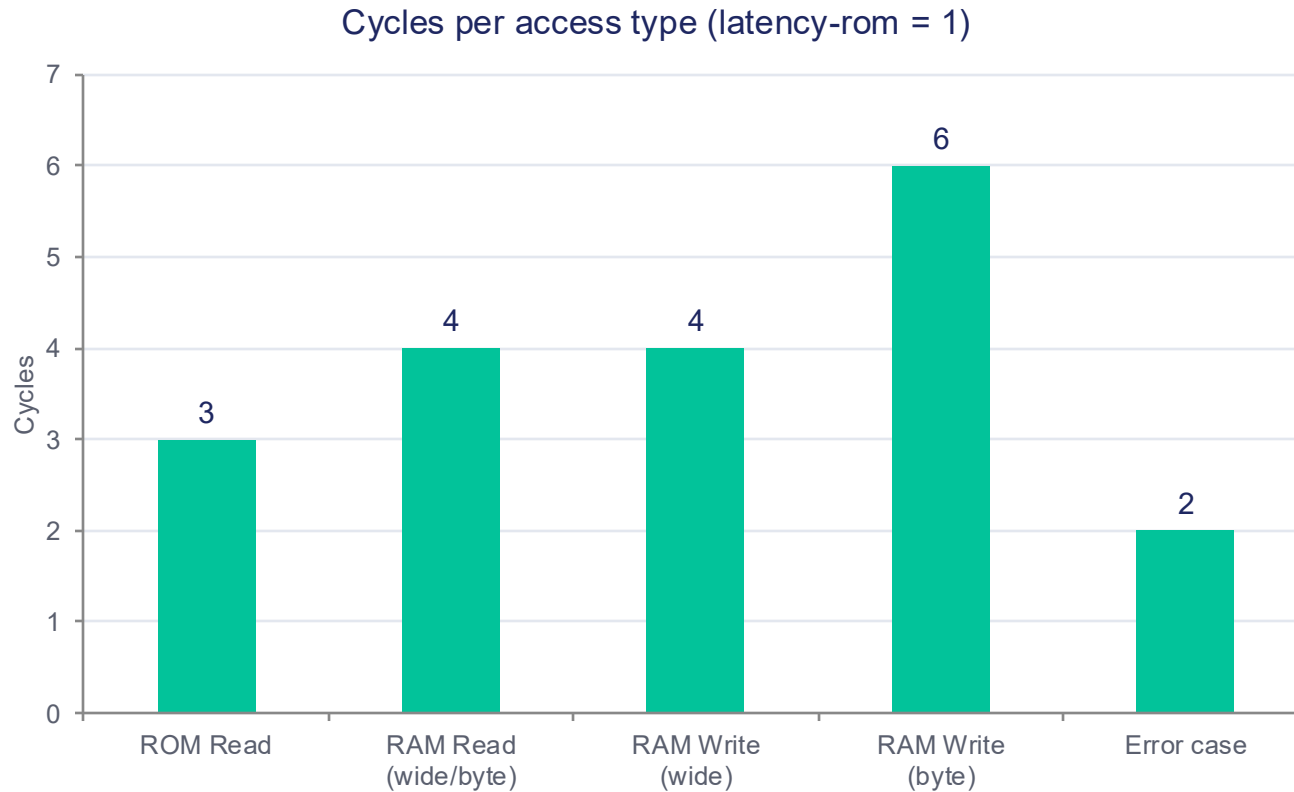




Evaluation

Andrei

Cycle cost per access type (own measurement)



Interpretation

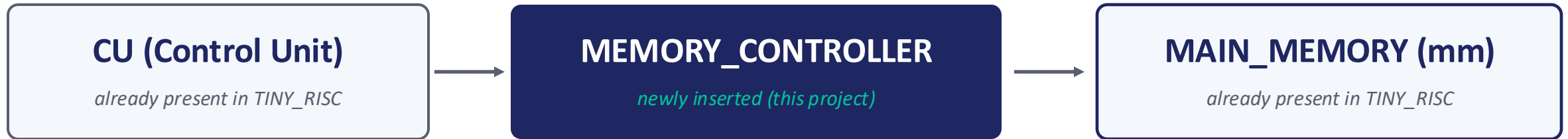
- RAM accesses cost +1 cycle vs. ROM (extra trip via mem_ready).
- 1-byte write accesses cost +2 cycles: a read-modify-write is needed to preserve the other 3 bytes.
- Error cases terminate immediately (2 cycles) — no main-memory access is needed.
- Comparison with reality: real memory controllers show the same effect for sub-word writes (e.g. DRAM/Flash) — read-modify-write overhead is a well-known pattern [3].

Measurement method: own simulation of the implemented module with isolated single requests, read out via the Result struct (cycles).

Integration into the TinyRISC CPU

Andrei

Based on the reference solution



This is how the signals between the CU and our controller would be connected — instead of directly to MAIN_MEMORY:

CU side (today)		MEMORY_CONTROLLER (new)	Meaning
cu.addr (via muxMemAddr)	→	addr	Target address for the access
cu.rmem	→	r	Read request active
cu.wmem	→	w	Write request active
cu.datain (via muxMemR)	←	rdata	Read value back to the CU
cu.memready	←	ready	Access completed
does not exist yet	→	wide, user	<i>Open point – the CU would need to be extended</i>
Controller ↔ MAIN_MEMORY stays unchanged from our own simulation: mem_addr, mem_wdata, mem_r, mem_w, mem_rdata, mem_ready.			

Summary

- Implemented Memory mapping, memory protection, and data-width adaptation
- C frontend: robust parsing & validation of CSV, ROM, and CLI input
- Nine targeted test scenarios cover all specified edge cases — all passing
- Cycle analysis shows the expected behavior (read-modify-write overhead on byte writes)

Outlook

- Finer-grained access rights (e.g. separate read/write permissions)
- A caching stage to reduce RAM access latency
- Parallelizing ROM and RAM accesses (Harvard-like separation)
- Integrating real TinyRISC signals and measuring the full system



Thank you for your attention!

Questions?

Niklas · Ahmed · Andrei



Sources

- [1] Leitenberger, B.: Die Harvard und von Neumann Architektur. <https://www.berndleitenberger.de/harvard-neumann.shtml>
- [2] Wikipedia: Memory Mapped I/O. https://de.wikipedia.org/wiki/Memory_Mapped_I/O
- [3] Edouard, H.: NOR vs NAND: So You Think You Know the Music? <https://www.jblopen.com/nor-vs-nand-so-you-think-you-know-the-music/>